

Day 3 Static Site Lab

이 lab은 이미 제공된 정적 웹앱을 Docker image로 만들고, container로 실행한 뒤, host에서 HTTP로 검증하는 실습이다. 새 파일을 즉석에서 만드는 실습이 아니라 **제공된 lab 파일을 읽고 운영 기준으로 실행하는 실습**이다.

File Map

파일	역할	확인할 점
index.html	nginx가 제공할 정적 HTML	화면에 표시될 문구, CSS 연결
styles.css	정적 페이지 스타일	COPY styles.css로 image에 포함되는지
Dockerfile	image build recipe	base image, WORKDIR, COPY, EXPOSE, CMD
.dockerignore	build context 제외 규칙	.env, log, dependency/cache 제외
README.md	실행/검증/runbook	다음 사람이 같은 결과를 재현할 수 있는지

Dockerfile 설명

현재 Dockerfile은 다음 계약을 가진다.

```
FROM nginx:1.27-alpine
WORKDIR /usr/share/nginx/html
COPY index.html ./index.html
COPY styles.css ./styles.css
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

줄	의미	운영 해석
FROM nginx:1.27-alpine	nginx 공식 image를 base로 사용	직접 웹서버를 설치하지 않고 검증된 base를 사용
WORKDIR /usr/share/nginx/html	이후 작업 기준 directory 지정	nginx의 기본 정적 파일 위치와 맞춤
COPY index.html ./index.html	HTML을 image 안에 복사	build context 안에 index.html이 있어야 함
COPY styles.css ./styles.css	CSS를 image 안에 복사	CSS 누락 시 페이지는 뜨지만 스타일이 깨질 수 있음
EXPOSE 80	container 내부 port 문서화	host port를 여는 것이 아님
CMD ...	nginx를 foreground로 실행	container가 바로 종료되지 않게 함

Preflight

repository root에서 시작했다면 lab directory로 이동한다.

```
cd week2/day3/labs/static-site
pwd
ls -la
```

Expected pattern:

```
.../week2/day3/labs/static-site
Dockerfile
.dockerignore
index.html
styles.css
README.md
```

Dockerfile과 제외 규칙을 먼저 읽는다.

```
sed -n '1,120p' Dockerfile
sed -n '1,120p' .dockerignore
```

불필요한 파일이 생겼을 때 .dockerignore가 왜 필요한지 확인한다.

```
mkdir -p __pycache__ node_modules dist build coverage tmp
printf "DO_NOT_COMMIT_TOKEN=example" > .env
printf "debug log" > app.log
printf "cache" > __pycache__/app.cpython-312.pyc
```

```
printf "large dependency placeholder" > node_modules/example.txt
printf "compiled output" > dist/app.bundle.js
find . -maxdepth 2 -type f | sort
rm -rf .env app.log __pycache__ node_modules dist build coverage tmp
```

이 예시는 실제 앱 실행에 필요 없는 파일이 얼마나 쉽게 lab directory에 섞이는지 보여준다. Dockerfile 이 COPY . .처럼 넓게 복사하는 구조라면 이런 파일이 image에 들어갈 수 있다.

Expected .dockerignore pattern:

```
.git
*.log
.env
.env.*
__pycache__/
*.pyc
.pytest_cache/
node_modules/
dist/
build/
coverage/
tmp/
```

대표 제외 대상:

패턴	이유
.env, .env.*	secret 노출 방지
__pycache__/, *.pyc, .pytest_cache/	Python cache 제외
node_modules/	큰 dependency directory 제외
dist/, build/, coverage/	build/test 결과물 제외
*.log, tmp/	실행 로그와 임시 파일 제외

Build

```
docker build -t paperclip-static-site:day3 .
```

Expected pattern:

```
[+] Building ...
=> [internal] load build definition from Dockerfile
```

```

=> [internal] load .dockerignore
=> [1/4] FROM docker.io/library/nginx:1.27-alpine
=> [2/4] WORKDIR /usr/share/nginx/html
=> [3/4] COPY index.html ./index.html
=> [4/4] COPY styles.css ./styles.css
=> exporting to image
=> naming to docker.io/library/paperclip-static-site:day3

```

성공 후 image 목록을 확인한다.

```
docker images paperclip-static-site
```

Expected pattern:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
paperclip-static-site	day3	<image-id>	<time>	<size>

Run

```
docker run -d --name paperclip-day3-static -p 18083:80 paperclip-static-site:day3
```

이 명령의 의미는 다음과 같다.

옵션	의미
-d	background에서 실행
--name paperclip-day3-static	container 이름 지정
-p 18083:80	host 18083을 container 80에 연결
paperclip-static-site:day3	실행할 image tag

container 상태를 확인한다.

```
docker ps --filter name=paperclip-day3-static
```

Expected pattern:

CONTAINER ID	IMAGE	STATUS	PORTS
<id>	paperclip-static-site:day3	Up ...	0.0.0.0:18083->80/tcp

Verify

host에서 HTTP header를 확인한다.

```
curl -I http://localhost:18083
```

Expected pattern:

```
HTTP/1.1 200 OK
Server: nginx
Content-Type: text/html
```

본문도 확인한다.

```
curl -s http://localhost:18083 | head
curl -s http://localhost:18083 | grep "Day 3 Static App"
```

Expected pattern:

```
<h1>Day 3 Static App</h1>
```

Security Scan

Docker Scout가 사용 가능한 환경이면 image 취약점을 점검한다. Docker 공식 CLI 기준으로 docker scout cves는 image 같은 software artifact의 CVE를 분석한다.

```
docker scout version || true
docker scout cves paperclip-static-site:day3 || true
docker scout cves --only-severity critical,high paperclip-static-site:day3 || true
```

Expected interpretation:

Scout 실행 가능: CVE summary, package, severity를 확인한다.
critical/high CVE 없음: 안전한 image 후보로 기록한다.
critical/high CVE 있음: base image 변경, package update, 예외 사유 기록 중 하나를 판단한다.
Scout 실행 불가: Docker Scout 미설치/로그인 필요/네트워크 문제를 blocker로 기록한다.

Scan note template:

```
Image: paperclip-static-site:day3
Scanner: Docker Scout / not available
Result: no critical/high / critical-high found / scan blocked
Action: accept / change base image / investigate / exception note
Reason:
```

Inspect Evidence

image가 어떻게 만들어졌는지 확인한다.

```
docker history paperclip-static-site:day3
```

Expected pattern:

IMAGE	CREATED BY	SIZE
<layer>	CMD ["nginx" "-g" "daemon off;"]	0B
<layer>	EXPOSE map[80/tcp:{}]	0B
<layer>	COPY styles.css ./styles.css	...
<layer>	COPY index.html ./index.html	...

metadata를 확인한다.

```
docker image inspect paperclip-static-site:day3 --format "{{.Id}} {{.Size}}
{{.Architecture}} {{json .RepoTags}}"
```

Expected pattern:

```
sha256:<id> <size> amd64 ["paperclip-static-site:day3"]
```

Cache Check

source를 조금 바꾼 뒤 rebuild한다.

```
printf '\n<!-- rebuild check -->\n' >> index.html
docker build -t paperclip-static-site:day3-v2 .
```

Expected pattern:

```
=> CACHED [1/4] FROM docker.io/library/nginx:1.27-alpine
=> CACHED [2/4] WORKDIR /usr/share/nginx/html
```

```
=> [3/4] COPY index.html ./index.html
=> [4/4] COPY styles.css ./styles.css
```

index.html을 바꿨기 때문에 COPY index.html 이후 layer가 다시 만들어질 수 있다. 이때 cache는 단순 속도 기능이 아니라 어떤 변경이 image build에 영향을 줬는지 보여주는 증거다.

Size Comparison

같은 앱이라도 base image 선택에 따라 image size가 달라진다. 비교용 Dockerfile은 Dockerfile.size-compare다.

```
sed -n '1,120p' Dockerfile.size-compare
```

```
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable -t
paperclip-static-site:size-default .
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable-
alpine -t paperclip-static-site:size-alpine .
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable-
trixie -t paperclip-static-site:size-trixie .
```

결과를 비교한다.

```
docker images paperclip-static-site --format "table {{.Repository}} {{.Tag}}
{{.Size}}"
docker image inspect paperclip-static-site:size-default --format "default
bytes={{.Size}}"
docker image inspect paperclip-static-site:size-alpine --format "alpine bytes=
{{.Size}}"
docker image inspect paperclip-static-site:size-trixie --format "trixie bytes=
{{.Size}}"
```

Expected interpretation:

같은 index.html/styles.css를 복사해도 base image에 따라 최종 size가 달라진다. Alpine 계열은 대체로 작고, Debian 일반/trixie 계열은 더 클 수 있다. 정확한 숫자는 Docker tag, platform, build 시점에 따라 달라지므로 학생은 자기 결과를 기록한다.

Image size가 커질 때의 영향:

영향	설명
build 속도	큰 layer와 context는 build/export/cache invalidation을 느리게 한다.
push/pull 시간	registry, CI runner, Kubernetes node가 image를 주고받는 시간이 늘어난다.
network 비용	여러 환경에서 반복 pull하면 traffic 비용이 증가할 수 있다.
storage 비용	registry와 node image cache 사용량이 증가한다.
rollout 시간	새 node가 image를 받는 시간이 길어져 배포가 늦어진다.
보안 scan	포함 패키지가 많으면 scan 시간과 취약점 표면이 늘 수 있다.

Failure Drill

1. Missing file

```
cd /mnt/d/paperclip
cp -r week2/day3/labs/static-site week2/day3/labs/static-site-broken
rm -f week2/day3/labs/static-site-broken/index.html
cd week2/day3/labs/static-site-broken
docker build -t paperclip-static-site:broken . || true
```

Expected pattern:

```
COPY index.html ./index.html
failed to calculate checksum
"/index.html": not found
```

해석: Dockerfile 문제가 아니라 COPY source가 build context 안에 없어서 생긴 build 단계 실패다.

2. Wrong port

```
cd /mnt/d/paperclip
docker run -d --name paperclip-day3-static-wrong -p 18084:8080 paperclip-
static-site:day3 || true
curl -I http://localhost:18084 || true
docker ps -a --filter name=paperclip-day3-static-wrong
```

Expected pattern:

```
curl: (52) Empty reply from server
# 또는 connection/reset 계열 실패
PORTS 0.0.0.0:18084->8080/tcp
```


해석: nginx는 container 내부 80에서 뜨는데 host port를 container 8080에 연결했기 때문에 verify 단계에서 실패한다. 수정은 -p 18084:80 또는 기존 정상 명령 -p 18083:80이다.

3. Wrong CMD

```
cd /mnt/d/paperclip/week2/day3/labs/static-site
awk '{ if ($1 == "CMD") print "CMD [\"nginx-bad-command\"]"; else print $0 }'
Dockerfile > Dockerfile.bad-cmd
docker build -f Dockerfile.bad-cmd -t paperclip-static-site:bad-cmd .
docker run -d --name paperclip-day3-bad-cmd paperclip-static-site:bad-cmd ||
true
docker ps -a --filter name=paperclip-day3-bad-cmd
docker logs paperclip-day3-bad-cmd --tail 30 || true
```

Expected pattern:

```
Exited (127)
exec: "nginx-bad-command": executable file not found in $PATH
```

해석: build는 성공했지만 container 시작 command가 잘못되어 run 단계에서 실패했다.

4. Bloated context

```
cd /mnt/d/paperclip/week2/day3/labs/static-site
mkdir -p node_modules __pycache__ dist build coverage tmp
printf "DO_NOT_COMMIT_TOKEN=example" > .env
printf "large dependency placeholder" > node_modules/example.txt
printf "compiled output" > dist/app.bundle.js
find . -maxdepth 2 -type f | sort
du -sh .
sed -n '1,160p' .dockerignore
rm -rf .env node_modules __pycache__ dist build coverage tmp Dockerfile.bad-cmd
```

Expected pattern:

```
.env
node_modules/example.txt
dist/app.bundle.js
```

해석: build가 실패하지 않아도 secret, dependency, build output이 context에 섞이면 보안/성능/비용 문제가 된다.

Tag Gate

```
docker tag paperclip-static-site:day3 paperclip-static-site:day3-reviewed
docker tag paperclip-static-site:day3 paperclip-static-site:v1.0.0
docker tag paperclip-static-site:day3 paperclip-static-site:staging
docker images paperclip-static-site
```

Expected pattern:

```
paperclip-static-site    day3
paperclip-static-site    day3-reviewed
paperclip-static-site    v1.0.0
paperclip-static-site    staging
```

docker tag는 새 build가 아니다. 기존 image에 사람이 읽기 쉬운 reference를 하나 더 붙이는 명령이다.

Tag를 붙일 때는 다음 기준을 먼저 정한다.

기준	예시	의미
앱 버전	v1.0.0	웹 애플리케이션 릴리즈 버전과 image를 연결
환경	dev, staging, prod	현재 어느 환경에서 쓰는지 표시
latest	latest	편의용 기본 tag, 운영 재현성 기준으로 단독 사용 금지
빌드 번호	build-128	CI 실행 결과와 연결
git sha	sha-a1b2c3d	source commit 추적
검수 상태	day3-reviewed, scan-passed	수업/검토 상태 표시

Version tag는 가능하면 웹 애플리케이션의 실제 버전과 맞춘다. 회사마다 semantic version, calendar version, sprint/release train 등 정책은 다를 수 있지만, image version이 앱 version과 분리되면 장애 분석과 rollback 때 추적 비용이 커진다.

Registry Push Gate

Docker Hub 또는 registry push는 선택이다. 다음 조건을 설명하지 못하면 push하지 않는다.

1. repository가 public인지 private인지 안다.
2. image context에 `.env`, token, 개인 파일이 들어가지 않았음을 확인했다.
3. tag 이름이 앱 버전, 환경, 빌드 출처, 검수 상태 중 무엇을 표현하는지 설명한다.
4. version tag는 웹 애플리케이션의 실제 릴리즈 버전과 맞춘다.
5. credential/token/MFA가 README, screenshot, terminal output에 남지 않는다.

Cleanup

```
docker stop paperclip-day3-static paperclip-day3-static-wrong paperclip-day3-  
bad-cmd || true  
docker rm paperclip-day3-static paperclip-day3-static-wrong paperclip-day3-  
bad-cmd || true  
rm -rf /mnt/d/paperclip/week2/day3/labs/static-site-broken
```

필요할 때만 image를 삭제한다.

```
# docker image rm paperclip-static-site:day3 paperclip-static-site:day3-v2  
paperclip-static-site:day3-reviewed paperclip-static-site:v1.0.0 paperclip-  
static-site:staging paperclip-static-site:size-default paperclip-static-  
site:size-alpine paperclip-static-site:size-trixie paperclip-static-  
site:broken paperclip-static-site:broken-fixed paperclip-static-site:bad-cmd
```

Completion Evidence

학생은 마지막에 다음을 남긴다.

- image tag:
- build success evidence:
- run command:
- HTTP verify result:
- vulnerability scan result:
- scan action or blocker:
- history/inspect evidence:
- failure drill type:
- fix and recheck:
- cleanup result: